The Copilot Ecosystem: GitHub, Microsoft 365 & Local Context

One brand family, three distinct operational contexts, different sources of truth and trust boundaries.

THE 10/90 AXIOM IN THE COPILOT ECOSYSTEM

Calling the foundation model is only **10% of the architecture**.
The remaining **90%** is authoritative context grounding (Microsoft Graph / Git workspace), permission trimming, action boundaries, and mandatory human verification.

> **Engineering principle:** The durable question is not "Which Copilot button do I click?" It is *"Where does the authoritative truth live, and what actions is the assistant allowed to take?"*

90-MINUTE LECTURE AGENDA

- **Copilot Taxonomy:** code delivery (GitHub), knowledge work (M365), and device (Windows)
- **Grounding Architecture:** Microsoft Graph, Semantic Index, and Enterprise Data Protection
- **Prompt Standard:** Microsoft Learn 4 Ingredients (Goal, Context, Source, Expectations)
- **GitHub Copilot in Depth:** Ghost text, IDE Chat, Agent Mode, and CLI
- **Live Demo (Project Northstar):** cross-surface business workflow in action
- **Practical Artifact:** constructing your personal Copilot Operating Map

INSTRUCTOR PROFILE • DELIVERY TRACK RECORD

Ihar Rubanovich (Igor Rubanovich)

Engineering Manager II & AI Ambassador @ EPAM Systems. Practitioner of AI-assisted engineering and production SDLC.

PROFESSIONAL FOCUS

- 10+ years in distributed systems engineering and delivery management.
- Systemic deployment of AI-assisted toolchains: CodeMie, Copilot, Cursor.
- Verified enterprise delivery: **EPAM CodeMie @ Dawn Foods** — 35% SDLC acceleration across feature delivery, unit testing, and architectural docs.

HANDS-ON AI R&D (PET-PROJECT)

Creator and architect of **Creator Tools** — a suite of AI tools for YouTube creators designed to save production time, streamline content workflows, and increase earnings.

HOMEWORK SUBMISSION & REVIEW PROTOCOL

- Official collaborator email for your GitHub homework repositories:

`ihar_rubanovich@epam.com`

- All assignments must be submitted via Pull Request to `genai-homeworks`.
- Every PR receives personal engineering code review from the instructor within one week.

CURRICULUM ROADMAP • 16 LESSONS • 6 MODULES

Where We Are: Module 3 (Engineer's Environment)

Core API fundamentals and multimodality are complete. Now we master developer environments and operational ecosystems.

MODULE 1 • DONE
Foundations & API
L01–L03: Tokens, Prompts, SDK

MODULE 2 • DONE
Multimodality
L04–L06: Images, Video, Audio

MODULE 3 • YOU ARE HERE
Engineer's Tools
L07–L09: Copilot, Data, Claude

MODULE 4 • PLAN
Local AI
L10: Ollama, LM Studio, RunPod

MODULE 5 • PLAN
Agents & MCP
L11–L13: Function Calling, MCP

MODULE 6 • PLAN
Production
L14–L16: Streamlit, Security

MODULE 3 SYLLABUS BREAKDOWN (LESSONS 07–09)

- **Lesson 07 (Today):** Copilot Ecosystem — context boundaries across GitHub, Microsoft 365, and Windows; the Operating Map.
- **Lesson 08:** AI for Data Analysis — Code Interpreter, sandboxed Python execution, profiling and data contracts.
- **Lesson 09:** Claude Ecosystem & Artifacts — shifting from conversational chat to workbench engineering, Claude Projects.

Homework Submission Protocol & Deliverable Standards

All assignments adhere to real-world software practices: git branches, pull requests, structured markdowns, and tests.

REPOSITORY TREE STRUCTURE:

```
genai-homeworks/
├── L01/ ...
├── L02/ ...
├── L07/
│   ├── copilot_operating_map.md    # 3-task operating map
│   ├── prompt_northstar_m365.txt   # 4-ingredient prompt
│   └── shipping_diff_review.md     # Diff review & test log
└── README.md
```

GRADING RUBRIC & ACCEPTANCE CRITERIA

1. Branch: `feat/L07-copilot-operating-map` off fresh `main`.
2. Invite `ihar_rubanovich@epam.com` as collaborator on GitHub.
3. PR description: clearly state the 3 real-world tasks selected and why each surface was chosen.
4. **Quality bar:** zero autonomy illusions — every task must specify a concrete human checkpoint and a durable evidence artifact.

The Evolution of Copilot: From Inline Tab to 2026 Ecosystem

Why "I use Copilot" is an underspecified statement without stating the runtime, data plane, and action scope.

2021 – 2022: GHOST TEXT

Smart Autocomplete

OpenAI Codex predicting lines in the current editor buffer. Context was limited to adjacent lines. Role: high-speed typing assistant.

2023 – 2024: CHAT & RAG

Conversational Surfaces

Sidebar chat in IDEs and Microsoft Graph grounding. Searching enterprise files via Semantic Index. Role: knowledge Q&A advisor.

2025 – 2026: AGENTS & ORCHESTRATION

Autonomous Workbenches

VS Code Agent Mode, test execution loops, MCP connectors, Copilot Studio, cloud PR agents, and NPU integration on Copilot+ PCs. Role: supervised autonomous agent.

TAXONOMY • ARCHITECTURAL SEPARATION

One Family, Three Worlds: Where Authoritative Context Lives

A core architectural rule: select the assistant surface by where the source of truth resides, not by brand logo.

TASK DOMAIN	COPILOT FAMILY	AUTHORITATIVE SOURCE OF TRUTH	TYPICAL PERMITTED ACTIONS	CONTROL PLANE
Software Delivery	**GitHub Copilot** (IDE, CLI, Cloud)	Git repo, issues, PRs, AST, test logs	Refactor, write tests, edit files, review PRs	GitHub Org/Enterprise policies, repository branch rules
Knowledge Work	**Microsoft 365 Copilot**	Microsoft Graph: Outlook, Teams, SharePoint, Word	Summarize meetings, draft briefs, analyze tables	M365 Admin Center, Entra ID, Purview DLP, ACLs
Local / Device	**Windows / Copilot+ PC**	Local screen, clipboard, OS files, NPU hardware	Search local files, analyze screen, Click to Do	Windows Group Policies, local user privileges

> **Golden Rule:** If your goal is to write a Python method, opening Microsoft 365 Copilot is useless (it cannot see your git tree). If your goal is to find executive meeting decisions, GitHub Copilot is helpless (it has no access to Teams or Outlook).

SYSTEMS ARCHITECTURE • C4 CONTAINER DIAGRAM

Grounding Architecture: How Enterprise Data Reaches LLMs

The foundation model does not know your company's files in advance — the orchestrator grounds the prompt prior to inference.

1. INTERCEPT & PRE-PROCESS

User Input

User submits a prompt in Teams or Word. The orchestrator extracts application context, user identity tokens, and kicks off semantic search.

2. GRAPH & SEMANTIC INDEX

Fact Retrieval

Queries Microsoft Graph. Semantic Index retrieves vector-similar documents from SharePoint/OneDrive strictly honoring the user's ACL permissions.

3. LLM ENRICHMENT

Inference Prompt

The LLM receives an augmented metaprompt: *User question + Grounded authoritative document excerpts + Output schema instructions*.

4. EDP & POST-PROCESS

Enterprise Protection

Output post-processing: sensitivity label filtering, PII redaction, source citations formatting. Customer data is never used to train foundation models.

Application Envelopes & Context Grounding Boundaries

Distinguish «Chat with Web» from «Grounded in Work Data». Each M365 application scopes available context differently.

COPILOT CHAT MODES

- **Web Grounded:** Public web search grounding. Cannot see company files. Ideal for external research and market analysis.
- **Work Grounded:** Connected to enterprise tenant. Accesses user's chats, emails, and accessible SharePoint documents.

> **UI Toggle Warning:** In Microsoft Copilot Chat, there is a visible toggle between Web and Work. Prompting about an internal project while set to Web will produce hallucinations or external web results!

IN-APP SKILL BOUNDARIES

- **Outlook:** Long thread summarization, draft replies grounded in meeting history.
- **Teams:** Instant meeting decision recaps from transcripts, unassigned action items.
- **Word:** Document synthesis from disparate meeting notes and source attachments.
- **Excel:** Formula generation and trend analysis (requires data formatted as an official Excel Table).
- **PowerPoint:** Transforming Word documents into slide deck narratives.

MICROSOFT LEARN METHODOLOGY • PROMPTING STANDARD

The 4 Prompt Ingredients for Copilot (Official Standard)

Microsoft Learn trains enterprise users to avoid vague requests and build structured four-part prompts.

```
[GOAL]:
Draft a structured rollout brief for Customer Support regarding the
new VIP Free Shipping rule.

[CONTEXT]:
We are releasing this feature to couriers and operators this weekend.
Support agents need clear qualification criteria and exception
workflows.

[SOURCE]:
Rely strictly on attached meeting_notes_fixture.docx and executive
decisions protocol.

[EXPECTATIONS]:
Format: Markdown with sections (Eligibility Criteria, Edge Cases, Open
Questions). Max 300 words. Include source paragraph citations.
```

ANATOMY OF THE 4 INGREDIENTS

1. Goal

Specific action verb: summarize, identify risks, format comparison table.

2. Context

Who will read this document and why the output is being produced.

3. Source

Explicit authoritative file, link, or meeting transcript (using `/` reference in chat).

4. Expectations

Strict constraints on tone, brevity, mandatory sections, and citations.

SECURITY & GOVERNANCE • ENTERPRISE RISK

Permission Trimming: Why AI Security Relies on Data Hygiene

Copilot does not invent a new access model — it strictly inherits existing user permissions via Zero Trust.

FEATURE: ZERO TRUST ACCESS BY DEFAULT

- If an engineer lacks access to the "Executive Comp 2026" SharePoint folder, Copilot can never retrieve or cite those files.
- User requests are processed within tenant boundaries with strict encryption.
- Complete isolation between organizations and restricted department sites.

CHIEF RISK: OVERSHARING IN SHAREPOINT

- Pre-AI, a sensitive spreadsheet left open to "Everyone in Company" sat unnoticed for years.
- **Copilot finds everything:** Any employee asking *"what are upcoming headcount changes?"* surfaces the document instantly via Semantic Index.
- Copilot rollout requires rigorous pre-deployment access audits in SharePoint and Teams!

ENTERPRISE DATA PROTECTION (EDP)

- **No Foundation Training:** Prompts and tenant data are never used to train foundation OpenAI/Microsoft models.
- **Data Residency:** Customer data remains within defined geographic boundaries (EU/US).
- **Purview Integration:** Native support for Sensitivity Labels, customer-managed keys, and comprehensive audit logs.

GITHUB COPILOT • DEVELOPER TOOLCHAIN

4 Operating Modes of GitHub Copilot

Distinguish synchronous editor completions, multi-file agent execution, and asynchronous cloud reviews.

1. GHOST TEXT

Inline Completions

Grey predictive text accepted via `Tab`. Triggers at cursor position from active buffer context. Fast boilerplate and loop generation.

2. IDE CHAT & SLASH

Sidebar Assistant

Chat participants: `@workspace` (entire repo index), `@terminal`. Slash commands: `/explain`, `/fix`, `/tests`, `/doc`.

3. AGENT MODE / EDITS

Multi-File Agent

Copilot Edits & Agent Mode. Autonomously refactors across multiple files, executes terminal tests, but requires human approval!

4. CLI & CLOUD AGENT

Terminal & PR Review

Terminal-native `copilot` CLI and background GitHub.com agents: automated Pull Request summaries and asynchronous code review.

What Copilot Sees in Your IDE: The Context Stack

Context in the code editor is not magic — it is an engineered stack assembled prior to each prompt completion.

VS CODE CONTEXT RETRIEVAL HIERARCHY

1. **Cursor & Active Buffer:** Code prefix and suffix around cursor.
2. **Neighbor Tabs:** Other files currently open in editor tabs.
3. **Repository Instructions:** `.github/copilot-instructions.md` in project root (enforces code style and constraints).
4. **Workspace Index:** Local embedding vector index of classes and methods.
5. **Configured MCP Tools:** External databases, APIs, and CLI runners.

SAMPLE `.github/copilot-instructions.md`:

```
- Always use explicit Python type annotations.
- Any modification to pricing logic requires unit tests in pytest.
- Preserve existing public API contracts and method signatures.
- Never hardcode fee thresholds or keys; read from config objects.
- Summarize proposed plan before applying code modifications.
```

LOCAL DEVICE • WINDOWS ECOSYSTEM

Windows Copilot vs Copilot+ PC: Cloud vs On-Device NPU

Distinguish the web application assistant from on-device AI capabilities accelerated by local NPU hardware.

CLOUD COPILOT APP ON WINDOWS

- Standard web/desktop Progressive Web App (PWA).
- Voice interaction, screenshot reasoning, Windows settings assist.
- Requires internet connection and Microsoft / Entra account.
- Processed on remote cloud datacenter clusters.

LOCAL COPILOT+ PC FEATURES (40+ TOPS NPU)

- **Recall:** Local semantic photographic timeline (runs entirely on-device with pause, filter, and encryption controls).
- **Click to Do:** Instant context actions on on-screen text/images without sending frames to third-party clouds.
- **Live Captions & Studio Effects:** 0ms latency hardware audio translation and webcam video processing.

> **Regional & Policy Variations (EEA vs US):** Depending on regulatory environments (e.g., EU DMA) and group policies, certain features (Recall, Click to Do) may be disabled or withheld. Always inspect local device capabilities before planning a workflow!

CROSS-SURFACE SCENARIO • PROJECT NORTHSTAR

End-to-End Case: One Business Change Across Three Contexts

Business objective: launch VIP Free Shipping. How work flows sequentially across three different tools.

1. BUSINESS DECISION (M365)

Executive Meeting

Decisions captured in raw meeting notes. **Microsoft 365 Copilot** generates a clear Support Rollout Brief with explicit source citations and highlighted open questions.

2. REPO IMPLEMENTATION (GITHUB)

Code & Tests

A ticket is logged. **GitHub Copilot Agent Mode** updates the `VIPShippingCalculator` in Python, writes pytest unit tests, and verifies via terminal test suite.

3. LOCAL RELEASE CHECK (WINDOWS)

Staging Verification

Developer validates local `release_checklist.txt`, staging feature flag toggles, and workstation preflights before triggering production deployment.

LECTURE MIDPOINT • COGNITIVE RESET

5-Minute Recharge Break: Rest Eyes & Coffee Refill

Strict rule: absolutely no work tasks or chat questions during break. Reset your attention before the live demo.

05:00

Coming up in Part 2: Live Demo of Microsoft 365 Copilot on fixture notes, VS Code Agent Mode with automated pytest verification, production disaster analysis, and the homework template.

HANDS-ON PRACTICE • LIVE DEMO PART 1

Demo 1: Microsoft 365 Copilot — Synthesizing the Brief

Processing meeting_notes_fixture.docx to draft a Support Rollout Brief using the 4 prompt ingredients.

DEMO EXECUTION WORKFLOW

1. Open Word or M365 Copilot Chat.
2. Attach fixture notes document using `/` reference.
3. Execute 4-ingredient prompt (Goal, Context, Source, Expectations).
4. **Verification Checklist:**

- Is the $50 order threshold for free shipping preserved?
- Are open questions highlighted (e.g., refund handling on delivery failure)?
- Are explicit paragraph citations present?

EXPECTED EVIDENCE ARTIFACT

Generated document `m365_expected_brief.md` :

- Unambiguous eligibility rules for Tier-1 support staff.
- Zero fabricated policy clauses missing from meeting notes.
- Ready for formal stakeholder sign-off.

HANDS-ON PRACTICE • LIVE DEMO PART 2

Demo 2: GitHub Copilot Agent Mode in VS Code

Implementing business logic in sample_repo: reading ISSUE.md, planning, editing code, and running tests.

VS CODE AGENT MODE PROMPT:

```
@workspace Read ISSUE.md and rules in .github/copilot-instructions.md.
1. Propose an implementation plan for VIP Free Shipping.
2. Implement calculation method in sample_repo/shipping.py.
3. Add boundary unit tests in sample_repo/test_shipping.py.
4. Execute smoke_check.py and confirm all tests pass.
```

DEVELOPER CHECKPOINT CONTROLS

- **Never trust conversational claims:** When the assistant reports "all tests pass", inspect actual terminal exit codes!
- **Diff Inspection:** Review side-by-side git diffs before clicking "Accept" to verify existing checks weren't deleted.
- **Terminal Approvals:** Only approve shell commands whose purpose and arguments you understand.

PRODUCTION REALITY • DISASTER ANALYSIS

Production Chronicles: 4 Catastrophic Failures vs Enterprise Win

Lessons learned from real-world enterprise incidents when deploying Copilot without boundary controls.

4 CATASTROPHIC INDUSTRY FAILURES

1. **Executive Salary Leak via Semantic Index:** Permissive HR permissions allowed Copilot to quote executive bonuses to junior staff.
2. **Rogue Terminal Flag in CLI:** Developer executed an AI-generated command containing a hallucinated flag that wiped staging state.
3. **Package Hallucination Exploit:** Coding assistant suggested a nonexistent npm package, which attackers registered with malware.
4. **Blind Test Acceptance:** Engineer merged agent PR without running local test suite; broke production with syntax errors.

ENTERPRISE SUCCESS: EPAM CODEMIE @ DAWN FOODS

- **35% SDLC Acceleration:** Systematic integration of AI engineering tools across Dawn Foods delivery streams.
- **Keys to Success:** Strict repository instructions, standardized task templates, context isolation, and mandatory human peer review before main branch merges.

DECISION FRAMEWORK • OPERATIONAL EVALUATION

Copilot Decision Matrix: 4 Evaluation Checkpoints

A practical checklist before kicking off any task: select the right surface and enforce safety boundaries.

4 QUESTIONS OF THE OPERATING MAP

1. Where is the authoritative source of truth?

Repository (code), M365 Graph (documents/meetings), or local OS state.

2. What are the permitted assistant actions?

Advise only, draft files, execute test suites, or create branch/PR.

3. What is the mandatory human checkpoint?

Code diff inspection, running unit tests, source link fact-checking.

4. What durable evidence artifact is retained?

Pull Request, SharePoint versioned doc, CI test pass log.

When Autonomous AI is STRICTLY FORBIDDEN (Red Zone):

- Executing financial transactions or issuing customer refunds.
- Direct production deployments bypassing CI and human sign-off gates.
- Processing sensitive PII or health data on surfaces lacking verified Enterprise Data Protection.

HOMEWORK ASSIGNMENT • PRACTICAL DELIVERABLE

Homework Assignment: Building a Copilot Operating Map

Analyze 3 real-world tasks from your daily engineering workflow and construct a governed Operating Map.

OPERATING MAP MATRIX (3 ROWS)

For each of your 3 chosen tasks, specify:

- **Task:** Concrete unit of work (e.g., "Refactor Glovo Courier dispatch fee logic").
- **Surface:** GitHub VS Code Agent Mode / M365 Word / Copilot CLI.
- **Context Source:** Git branch, Jira issue description, meeting transcript.
- **Allowed Action:** Edit local files and run unit test suite.
- **Human Checkpoint:** Inspect diff and verify > 90% test coverage.
- **Evidence:** Link to Pull Request with passing CI run.
- **Policy Dependency:** Corporate restriction against unvetted MCP servers.

MANDATORY 4-INGREDIENT PROMPT

For one of your tasks, write a production-grade prompt adhering to the Microsoft standard:
[Goal] + [Context] + [Source] + [Expectations].

Commit `copilot_operating_map.md` to `L07/` in your `genai-homeworks` repository.

WRAP-UP • KEY TAKEAWAYS & LESSON 08 TEASER

Key Takeaways, Open Mic & Next Session Teaser

4 core takeaways from today's session. Microphones are now open for Q&A.

4 CORE LESSON TAKEAWAYS

1. **Copilot is a Portfolio:** Choosing the right tool depends on where authoritative truth lives.
2. **Context Trumps Model:** Grounding via Graph or Git workspace determines output quality.
3. **The 4 Ingredients:** Goal + Context + Source + Expectations eliminates 95% of hallucinations.
4. **Verification is Mandatory:** Code diffs, test logs, and citations are your only defense against errors.

NEXT LESSON • L08: AI FOR DATA ANALYSIS

Data Analysis & Code Interpreter:
Why should language models never calculate numbers in their heads? How to use sandboxed Python execution to compute business metrics, validate data contracts, and generate trustworthy charts.

Open mic — ask questions in chat or voice!